

ADAMKYC React.js App

Introduction

Hello. DeltaPlata welcomes you!

We will look at a project called “ADAMKYC React.js App”. As the name suggests, this is a plugin that is built into a page in the WordPress environment, thus allowing you to perform additional functionality (search, creation of entities, advanced search, system for uploading data to the server, working with tables).

The goal of the project is to complement the functionality of the linked parent page, thus increasing the functionality. It is necessary to ensure efficient and minimalistic design, competent code structure, efficiency and speed of query execution. During the writing of the project, all goals were achieved.

Project structure

The project structure is built as follows:

1. github - GitHub Actions main entry point
 - a. workflows - deploy scripts, test script
2. husky - lints your commit messages
3. devops - building logic
 - a. build.sh - read Deployment step for more info
4. public - contains public files, such as index.html
5. .env.example - environment file example, contains all backend properties which is required to be able to connect
6. .gitignore - ensures that certain files are not tracked by Git
7. jsconfig.json - config file, that contains import logic
8. package-lock.json - ensures that the same dependencies are installed consistently across different environments
9. package.json - contains descriptive and functional metadata about a project
10. README.md - short explanation on how to get started

Source Diretory (src)

1. api - contains all files, that responsible to backend connection
2. assets - contains all SVG images that are used in the project.
Contains index.js file, which provides dynamic import logic.
3. components - components folder
 - a. Admin - admin components folder

- b. AdminAsideButton - component, that used in AdminEntityCreate Page aside component
- c. AdminButton - reusable component that used in most Admin Page views
- d. AdminControlButton - reusable component that is used in most Admin Page tables. Provide control logic
- e. AdminEditProfile - folder, that contains all components, used in AdminEdit / AdminCreate Page
- f. Analytics - folder, that contains all components for Analytics Page
- g. Details - folder, that contains all components for AdminEdit Page
- h. Bulk - reusable component, show the ADS information, and lead to Advanced Search Page
- i. BulkCategorySelector - component, contains a Array of selectors (buttons) to select uploading category
- j. BulkSampleDownloader - component, that allows user to download sample, for how and in which format he must upload files to backend
- k. BulkUploadedHistory - component to display uploaded files to backend, in table format
- l. BulkUploader - component for downloading files to backend
- m. Button - reusable button component
- n. Card - reusable component for Search Page
- o. Container - component - layout for children elements
- p. CountryFilter - reusable country - select component
- q. CustomAlert - reusable component for display errors, notifications, hints
- r. Database - the only unique component, and at the same time page itself. Contains login / disconnect logic to DB
- s. DateFilter - reusable date - input component
- t. Details - folder, that contains required components for DetailsPage
- u. DownloadBODS - reusable REPORT download button component
- v. Filter - reusable select-filter component
- w. FormGroup - reusable form component
- x. Forms - contains all form components
- y. Graphs - contains all graph components
- z. HashView - reusable component for hashtag render logic
- aa. Header - reusable component, that used in layouts
- bb. Hero - component, that used as <main> in Welcome Page
- cc. NotFound - reusable component, that showing in case of no results, or wrong URL
- dd. Paginator - reusable pagination component with original name
- ee. QuickAction - reusable 3 dot styled component, used in case if we need to modify Entity on tables
- ff. ReportIncorrectData - component, that is responsible for working with report logic (backend + frontend)
- gg. Select - reusable select - input component
- hh. ShareButton - reusable share button component, for search and Entity Page

- ii. Sort - reusable select - input component
- jj. SpecialFilter - component, that contains checkboxes with possible search properties (categories)
- kk. Spinner - reusable component, which you can see, when the pages making fetch or loading some logic
- ll. Table - folder, that contains all possible Tables in frontend part
- mm. TableOfContent - Entity Page navigation aside component, can redirect to any showed section
- nn. Title - reusable title - layout component
- oo. TitleWithTooltip - reusable text with hint component
- pp. App.jsx - Routing and Pages middleware connector
- qq. index.js - re-import file for all components
- 4. helpers - folder for all used helper - functions. To decrease logic in components / Pages
- 5. layouts - folder, that contains all layouts for routes
- 6. pages - folder, that contains all 'VIEWS' files
 - a. Admin - folder, that contains all Admin Pages
 - i. AdminAnalyticsPage - Admin Analytics Page
 - ii. AdminBulkUploadDataPage - Admin Entities Table Page
 - iii. AdminBulkUploadPage - Admin Uploading Page
 - iv. AdminBulkUploadResultsPage - Admin Uploaded Table Page
 - v. AdminEditProfileDetailPage - Admin Create/Updating Entity Page
 - vi. AdminEditingRequestDetailPage - Admin Report Edit / Create Page
 - b. AdvancedSearchPage - Advanced Search Page
 - c. DataUploadPage - File uploading Page for users
 - d. DataUploadResultsPage - Uploaded user files Table Page
 - e. DetailsPage - Entity View Page
 - f. ReportIncorrectDataPage - Report Incorrect Data Page
 - g. SavedResultsPage - Bookmarks Page
 - h. SearchHistoryPage - Search History Page
 - i. SearchPage - Search Page
 - j. WelcomePage - main (HOME) Page
- 7. providers - folder, that contains all <Context> providers, and they logic
- 8. styles - folder, that contains main.scss file
- 9. index.css - main css file, have :root inside of itself
- 10. index.js - main js file, connects App logic

Technologies

This project uses basic layout technologies and has dynamic logic on pages.
The following were used:

- HTML 5
- CSS
- SCSS

- Javascript
- Chart.js
- React.js with React Router
- Husky

Everything is written by hand, and libraries are hardly used. In this way, it was possible to increase initial page loading speed, save a lot of memory, optimize the project, and also adapt many elements to the project.

Setup

To install the project you need to use a simple chain of commands:

1. Clone repository, or download project archive to your PC
2. Open terminal (Any, but Bash or Powershell recommended)
3. Execute `npm install` and wait until all dependencies are installed
4. Execute `npm run start` to run the project on localhost:3000

Testing

To test the project with a minimal set of tests (for errors during the build command), you can use `npm run test`, which runs the testing script. You can add your tests to workflows, and you can add them to the build script to automate the running of tests before building the project.

Deployment

You need to assemble the project into one entity that can be later embedded inside of WordPress Plugin. You can do so by running `bash devops/build.sh`. After that process completes just copy content of the **build/static** folder to WordPress plugin **static** folder.

Updates

To keep the project up-to-date, you must use the `npm update` command in the terminal after git pull, or use github desktop + fetch. You can also simply reload the project with a new archive, this way you will use the new updates. Don't forget that with the latter method, you will need to follow the installation instructions again.

Troubleshooting

If you find a bug, or witness irrational behavior of the application, we recommend contacting the product developers through contacts.

License

The project is protected by copyright, unauthorized use is prohibited and will be punished by law. If you have requirements or suggestions, you need to contact the development team to agree on your question.

Contacts

To contact the development team, please send your request by email to:

contact@deltaplata.com